

Data Quality Rule Generation with LLMs

Anna-Christina Glock
Software Competence Center
Hagenberg, Austria
anna-christina.glock@scch.at

Thomas Hütter
Software Competence Center
Hagenberg, Austria
Thomas.Huetter@scch.at

Johannes Fürnkranz
Johannes Kepler University
Linz, Austria
juffi@faw.jku.at

Wolfram Wöß
Johannes Kepler University
Linz, Austria
wolfram.woess@jku.at

Christine Dominka-Kiss
Austrian Post
Vienna, Austria
christine.dominka-kiss@post.at

Lisa Ehrlinger
Hasso Plattner Institute, University of
Potsdam
Potsdam, Germany
lisa.ehrlinger@hpi.de

Abstract

The validation of data, such as customer and employee data, is an important task in many organizations. Errors in data can have severe consequences. For example, a wrong drug unit in a patient record can lead to life-threatening medication errors, and a missing street number in an address to failed deliveries. Companies often employ rule-based enterprise data quality (DQ) tools, which allow domain experts to specify rules to validate the data over time. While rule-based DQ tools are computationally efficient and provide explainable reports, maintaining a comprehensive rule set manually is challenging, as domain experts often overlook essential rules, especially in complex domains and large data volumes. Hence, closing these gaps remains an open problem in practice.

In this paper, we address the challenge of automated DQ rule generation. For this, we formalize a generalizable *generate-filter framework* and introduce LEDQER, an LLM-based DQ rule generation approach. First, a large language model (LLM) generates candidates rules from an observed dirty data tuple for a given rule-based DQ tool syntax. Second, we apply four filter techniques that ensure the (i) executability, (ii) correctness, and (iii) generalizability, and avoid (iv) redundancy of the generated rules. An extensive experimental evaluation suggests that LEDQER is able to produce effective and compact rule sets for various datasets and error types.

CCS Concepts

• **Information systems** → **Data cleaning**; • **Applied computing** → Business rules; Enterprise data management.

Keywords

Data quality, rule generation, rule-based tools, large language models

1 Introduction

Data quality (DQ) has long been recognized as a critical prerequisite for effective decision-making in organizations [32, 35, 40]. Haug et al. [18] show the impact of poor DQ on organizations that can range from incorrect decision-making and decreased customer satisfaction to life-threatening medication errors in healthcare [7]. As organizations increasingly deploy artificial intelligence (AI) in practice, the impact of poor DQ becomes even more severe: AI models trained on low-quality data show reduced performance [26, 27].

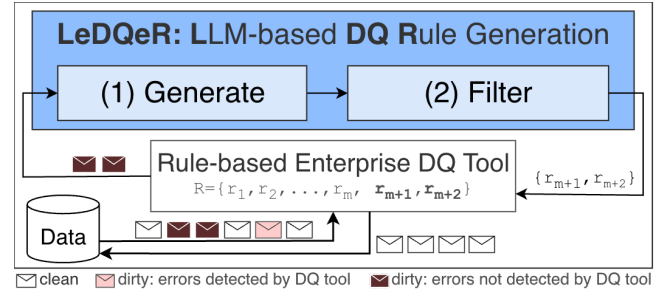


Figure 1: LEDQER extends the rule set of a rule-based enterprise DQ tool. Incoming data is validated against an existing rule set $R = \{r_1, \dots, r_m\}$. Errors covered by a rule are detected, while errors not covered by any rule pass the validation unnoticed and cause failures in downstream processes. LEDQER closes these gaps: undetected dirty tuples are passed to the generate-filter pipeline, which produces new rules $\{r_{m+1}, r_{m+2}\}$ that extend the existing rule set such that the same error type is detected in the future.

Beyond these technical considerations, DQ has also become a regulatory obligation: the EU AI Act mandates that training, validation, and test data of high-risk AI systems must be *relevant*, *representative*, *free of errors*, and *complete* [10]. Consequently, even in the era of AI and large language models (LLMs), detecting and correcting data errors in enterprise settings remains an essential requirement.

Machine learning (ML)-based approaches to automate DQ tasks, such as error detection and correction, with minimal expert input have been proposed in research [e.g., 25, 37]. However, these tools lack vendor support and system integration capabilities that are often required by large organizations, and are hence rarely deployed in enterprise settings. More recently, also LLMs have been applied directly to detect errors in data [3, 4, 14, 29, 43]. While effective especially for semantic and context-dependent error types, these approaches require passing the data through the LLM, which is computationally expensive at the scale of enterprise data and typically conflicts with strict data protection requirements, such as the European General Data Protection Regulation (GDPR) [9], when the data contains personal information. Moreover, Bodensohn et al. [3] show that the accuracy of LLMs declines sharply on real-world

enterprise data, e.g., due to large table sizes and missing internal knowledge.

Consequently, organizations predominantly rely on enterprise DQ tools [6, 8], which are largely rule-based solutions with some ML and AI support. These tools are computationally efficient, produce deterministic and interpretable results, and are well-established in production. While some enterprise DQ tools recently integrate LLMs to translate natural-language descriptions into rules [36], the rule creation itself remains a manual process: a data steward still needs to know and specify which rules are required.

Problem statement. However, a rule-based DQ tool is only as good as its rule set. Figure 1 illustrates this setting: incoming data is validated against an existing rule set $R = \{r_1, \dots, r_m\}$. Data errors that are covered by a rule are detected and can be handled accordingly. Data errors that are not covered by any rule, however, pass the validation unnoticed. As a consequence, the data is classified as clean and propagates into downstream processes, such as ETL pipelines or master data management, where the undetected errors cause processing failures. Consider our industry use case at the Austrian Post: a missing street number or a semantically hard error, such as the confusion of the Austrian cities Lienz and Linz, can lead to failed deliveries. Such failures are particularly costly because they surface late and reoccur until the rule set is extended accordingly. Since manually defining and maintaining DQ rules is complex and time-consuming [5, 8, 39], closing these gaps in the rule set remains an open problem in practice.

Our approach. In this paper, we solve this problem with LEDQER, a LLM-based framework that iteratively extends the rule set of an enterprise DQ tool whenever an undetected error is identified (cf. Figure 1): the dirty tuple that caused a downstream failure is passed to LEDQER, which generates DQ rules that detect the error such that the same error type cannot pass the validation unnoticed again. LEDQER follows a generate-filter approach. In the *generate* step, a LLM automatically generates candidate DQ rules from a dirty tuple and a given DQ rule format. In contrast to traditional rule-learning methods [5, 44], which struggle to capture semantic context and frequently require labeled data, LLMs leverage pre-trained knowledge and semantic reasoning capabilities to generate DQ rules in code-like syntax (e.g., SQL or Python) from minimal input [33, 42]. In the *filter* step, the candidate rules are progressively refined with respect to (i) executability, (ii) correctness, (iii) generalizability, and (iv) redundancy, producing a minimal and precise rule set that is ready for operational deployment.

Contributions. In summary, this paper makes the following contributions to automate DQ rule generation in enterprise settings:

- We formalize the problem of DQ rule generation and introduce a generalizable generate-filter framework.
- We propose LEDQER, a LLM-based implementation of the framework that generates DQ rules from observed dirty tuples and incrementally extends an existing rule set.
- We introduce a suite of four filter techniques that ensure the (i) executability, (ii) correctness, (iii) generalizability, and (iv) redundancy of the new rules.
- We comprehensively evaluate LEDQER on five datasets from diverse domains with up to nine different error types.

Results. Our evaluation shows that LEDQER enables the generation of executable rules that generalize well to unseen data without producing false positives. In particular, our experiments suggest that LLMs are promising for generating DQ rules and show the effectiveness of our filters. We also observed that the ability of the generated rules to detect a dirty tuple is influenced by the error type and the amount of error context provided in the prompt.

Outline. We first introduce necessary background on data errors and rule-based DQ tools in Section 2, where we also present the running example used throughout this paper. In Section 3, we first formalize the problem of DQ rule generation and subsequently propose LEDQER, an LLM-based framework for DQ rule generation that follows our generate-filter formalization. We evaluate LEDQER in Section 4 and discuss related work in Section 5. Section 6 concludes this paper with an outlook on future work.

2 Data Errors and Rule-based DQ Measurement

In this section, we introduce the necessary background on data error types in Section 2.1 and rule-based DQ tools in Section 2.2. Throughout this section, we will also introduce a running example that is derived from our industry use case at the Austrian Post: a personal contact information (PCI) dataset (cf. Table 1). This example serves as a walkthrough to explain our generate-filter framework in Section 3 and its experimental evaluation in Section 4.

2.1 Data Quality and Error Types

Table 1 shows our running example: a subset of a personal contact information (PCI) dataset from our industry use case at the Austrian Post. Each tuple describes a person and their address with the columns Street, FirstName, PostalCode, and City. The example is a reduced version of the full PCI dataset with 14 columns, which we use in our experimental evaluation (cf. Section 4).

Data quality is per definition context-dependent, as it is commonly defined as “fitness for use” [40], which means that the data must be fit for the specific purpose it is used for. Consequently, DQ measurement must reflect domain-specific semantics and usage expectations.

Example 2.1. The use case of our running example introduces the following requirements on the data: (1) street names must contain actual street names and no placeholder values such as “Unknown”, (2) a four-digit numerical representation like “1220” is required for the postal code, even though a leading “A” is commonly added in Austria (e.g., “A-1220”), and (3) the city solely consists of the city name, with no subareas being allowed. Values that violate these requirements are data errors and are highlighted in color in Table 1.

As illustrated in Example 2.1, data errors are observable effects of poor data quality and are inherently context- and usage-specific. This has led to the development of numerous data-error taxonomies [2, 19, 20, 23, 28, 30, 34]. Based on these taxonomies and feedback from domain experts, we selected nine error types that cover a broad range of DQ issues that appear in single values or tuples for evaluating our framework:

Explicit missing values (emv) are values explicitly set to null. This is a common error type [31] and well-known in literature [2, 19, 21, 23, 28, 30, 34].

Table 1: Subset of the PCI dataset as running example.

ID	Street	FirstName	PostalCode	City
1	Unknown	Ina-Maria	A-1220	Wien
2	Brahmsplatz	Hans	1040	Wien, Wieden
3	Berggasse	Anna	A-4020	Linz
4	Feldgasse	Anna-Maria	A6020	Innsbruck
5	Unknown	Maria	8010	Graz

Type of error: ■ Embedded Value
 ■ Incorrect Format ■ Disguised Missing Value

Disguised missing values (dmv) are semantically missing (like emv) but contain placeholder values. For example, John Doe as FirstName and LastName or 01.01.1900 as DateOfBirth. Detecting this error type is challenging because they might be syntactically valid but semantically incorrect [2, 31].

Contradictions (con) are mutually inconsistent values among different values of a tuple [28]. For example, both the postal code and the city may be syntactically correct, but do not match the same entity. Even though literature sometimes describes such errors under the umbrella of “functional dependency violations” [20, 30], practical scenarios can be more complex. For example, no functional dependency holds between city and postal code. In other words, one postal code can refer to multiple cities (e.g., 4232 is the postal code for 9 municipalities in Austria), whereas one city can have multiple postal codes (e.g., Vienna has 23 postal codes). Therefore, we use the more general term “contradiction” introduced by Müller and Freytag [28] in this paper.

Misfielded values (mfv) are correct values that are switched and therefore end up in the wrong field/column. For example, a value of a FirstName is stored in the respective LastName of the same tuple and vice versa. This error type has already been described by [2, 21, 23, 34].

Embedded values (ebv) are values that contain additional, unwanted information [2, 30, 34]. For example, the postal code “1220, Donaustadt” also contains the district name.

Spelling mistakes (spm) are incorrect values resulting from added, deleted, or exchanged characters/numbers [2, 20, 21, 23, 30, 34]. Common examples include typos, such as missing an “n” in “Viena”.

Domain violations (dov) refer to values that are outside of an expected range or violates business constraints [20, 28, 30]. For example, a negative PhoneNumber or the violation of a business rule that enforces the country column to contain the full name of the country.

Incorrect formats (ifo) may appear for various columns and data formats. For example, the date might follow different standards (‘MM/DD/YYYY’ vs. ‘DD/MM/YYYY’). This error type is listed as a syntax violation in [2, 30], as a domain format error in [28], and as a format error in [21].

Incorrect character sets (ics) are used in the data. For example, the Cyrillic character set was used to write the address instead of the Latin character set. This error type was also introduced by Kim et al. [23], who focus on wrong encodings (e.g., ASCII vs. EBCDIC) and also mentioned as incorrect encoding in [2, 21].

While not exhaustive, the selected error types capture the major classes of DQ issues described in literature and encountered by practitioners.

Example 2.2. Each tuple in Table 1 is identifiable via column ID and contains at least one dirty value, which can be classified as one of the error types: the errors in Street are of type dmv, as two tuples contain the value *Unknown*, which indicates a missing street name without actually being a missing value. FirstName has no errors. Three values in PostalCode are of error type ifo, since the correct format is the four-digit postal code as shown in tuples 2 and 5. Tuple 2 violates the requirement that the city is limited to the city name and is hence of type ebv.

2.2 Rule-based Data Quality Measurement

Rule-based DQ measurement [24] is a foundational approach to measuring data quality, relying on an explicitly defined rule set that captures the expected data properties. These rules formalize domain knowledge and business constraints (e.g., allowable value ranges, referential integrity conditions, or pattern-based formats) and are systematically evaluated against datasets to detect violations.

Several practical data quality tools and frameworks support the rule-based approach [8]. Prominent open-source systems include Great Expectations¹(GX), Amazon Deequ², and Apache Griffin³. In the commercial domain, platforms such as Informatica⁴, Talend Data Quality⁵, and Ataccama⁶ offer support for rule-based data quality evaluation. Despite differences in implementation details, these tools share a common architecture in which explicitly defined rules are evaluated against data to identify violations that subsequently provide data quality indicators. In the remainder of this paper, we focus on GX as a representative rule-based data quality framework due to its widespread adoption and open-source nature. A list of expectations (i.e., rules) in GX syntax is shown in Example 2.3.

Example 2.3. Based on the data in Table 1, a set of hypothetically generated, possibly incorrect rules is given below:

- (1) `gx.expectations.ExpectColumnValuesToNotMatchRegex(column='PostalCode', regex=r'A-[0-9]*')`
- (2) `gx.expectations.ExpectColumnValuesToMatchRegex(column='Street', regex=r'^Unknown$')`
- (3) `gx.expectations.ExpectColumnValuesToNotMatchRegex(column='FirstName', regex=r'^[A-Z][a-z\-\]*$')`
- (4) `gx.expectations.ColumnNotMatchRegex(column='Street', regex=r'^[A-Z][a-z]*')`

¹https://github.com/great-expectations/great_expectations

²<https://github.com/aws-labs/deequ>

³<https://griffin.apache.org/>

⁴<https://www.informatica.com/products/data-quality.html>

⁵<https://www.talend.com/products/data-quality/>

⁶<https://www.ataccama.com/platform/data-quality>

Each rule specifies a pattern with a regular expression, where the operation defines whether matching values are considered correct or incorrect. Rule (1) states that postal codes starting with 'A-' are incorrect, which correctly identifies the entries marked in red in Table 1. Rule (2) is an example for an incorrect rule because it expects street names to match the value "Unknown", thereby erroneously flagging the valid street names in tuples 2–4 while accepting the actual errors. Rule (3) states that first names that consist of a capitalized letter followed by lower-case letters, possibly separated by hyphens, are incorrect, which would erroneously mark entries 2, 3, and 4 of the first name column as errors. Rule (4) syntactically violates the GX format and is not executable at all. Note that these hypothetical rules are intentionally imperfect and illustrate typical problems that occur during rule generation. In Section 3, we show how our filters remove such rules.

While rule-based DQ frameworks provide deterministic and transparent data validation, there are some notable limitations: First, data stewards often lack complete insight into newly emerging or fast evolving data domains, which makes it difficult to define comprehensive rule sets upfront and maintaining hundreds of rules over time. Second, defining rules for semantic error types (e.g., "empty" refers to a missing value) is more difficult than for syntactic error types (e.g., Austrian postal codes must consists of four digits). In summary, the manual generation of suitable DQ rules remains a huge and practical challenge in enterprise settings.

3 LLM-based Data Quality Rule Generation

We introduce a LLM-based approach for data quality rule generation that addresses the shortcomings of manual approaches. In this section, we formalize the problem, introduce a general generate-filter framework for data quality rule generation, and then present our LLM-based approach to instantiate the framework.

3.1 Problem Formalization

Data quality rule generation aims at automatically deriving a set of DQ rules $\mathcal{R} = \{r_1, r_2, \dots, r_m\}$ for a given dataset $\mathcal{D} = \{t_1, t_2, \dots, t_n\}$ of n tuples and a rule-based DQ tool supporting rule syntax $\mathcal{S}$. The quality of the generated rules depends on their executability (i.e., syntactic correctness) and their ability to identify the corresponding error types in the data.

To foster the discussion of existing [38, 41], our proposed, and future approaches, we introduce a generalized framework for data quality rule generation, called *generate-filter framework* (cf. Figure 2). The framework produces a rule set $\mathcal{R}$ in two steps:

- (1) *Rule Generation*: Given an input dataset $\mathcal{D}$, the objective is to generate an initial set of candidate DQ rules $\mathcal{C}$.
- (2) *Rule Filtering*: Since the generated rule candidates $\mathcal{C}$ may include erroneous, redundant, or overlapping rules, a subsequent filtering phase is applied to identify and remove low-quality rules. Thereby, the overall correctness and coverage of the resulting rule set $\mathcal{R}$ is improved.

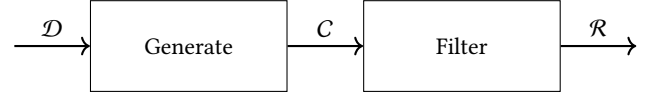


Figure 2: Generate-filter framework for DQ rule generation.

Example 3.1. Consider dataset $\mathcal{D}$ from Example 2.1. In the generate step, a set of $|\mathcal{C}| = 4$ candidate rules for the DQ tool GX is generated (cf. Example 2.3). Assume that in the filtering step, the syntactical correctness of the rules is verified. Consequently, $\mathcal{R}$ contains three rules (1-3), while rule (4) is discarded because it does not conform to GX's syntax.

3.2 LEDQER: An LLM-based Generate-Filter Framework to Generate DQ Rules

We introduce LEDQER, a LLM-based instantiation of the generate-filter framework. Figure 3 shows the architecture of LEDQER: First, we generate a candidate DQ rule set using a LLM-based technique. Second, we propose a set of four complementary filters that progressively refine the candidate rule set $\mathcal{C} \supseteq \mathcal{C}' \supseteq \mathcal{C}'' \supseteq \mathcal{C}''' \supseteq \mathcal{R}$.

3.2.1 LLM-based DQ Rule Generation. Instead of defining the set of DQ rules by hand, as it is typically expected by state-of-the-art DQ tools (cf. Section 2), we present an automated LLM-based approach to generate DQ rules. Algorithm 1 describes the generation process, which requires the following inputs: a (dirty) dataset $\mathcal{D}$, a LLM model $\mathcal{M}$, a prompt template $\mathcal{P}$, a DQ rule syntax $\mathcal{S}$, and a textual description $Desc$ of the semantic context of the dataset (e.g., “The data contain information about the scheduled and actual departure and arrival times of airplanes for a flight dataset.”). Each tuple $t_i \in \mathcal{D}$ contains at least one error. For each tuple t_i , a prompt template $\mathcal{P}$ is populated with the dirty record, a guideline for the DQ rule syntax $\mathcal{S}$, and an overall dataset description. For this prompt, the LLM generates a set of candidate rules $\mathcal{C}_i$ for tuple t_i . The number of generated rules $|\mathcal{C}_i|$ may differ from the actual error count of the dirty record, as the LLM is unaware of the exact number of errors and may generate multiple rules or none for a specific error. The global candidate set $\mathcal{C}$ is defined as the union of all individual rule sets $\mathcal{C}_i$ generated for each tuple t_i with $1 \leq i \leq n$, such that $\mathcal{C} = \bigcup_{i=1}^n \mathcal{C}_i$.

Algorithm 1 DQ Rule Generation

Input: $\mathcal{D}$ (Dirty Dataset), $\mathcal{M}$ (LLM Model), $\mathcal{P}$ (Prompt Template), $\mathcal{S}$ (Rule Syntax), $Desc$ (Dataset Description)

Output: $\mathcal{C}$ (Candidate set of DQ Rules)

```

1: procedure GENERATERULES( $\mathcal{D}, \mathcal{M}, \mathcal{P}, \mathcal{S}, Desc$ )
2:    $\mathcal{C} \leftarrow \emptyset$ 
3:   for  $t_i \in \mathcal{D}$  do
4:      $prompt \leftarrow \text{constructPrompt}(\mathcal{P}, t_i, \mathcal{S}, Desc)$ 
5:      $response \leftarrow \text{queryLLM}(prompt, \mathcal{M})$ 
6:      $\mathcal{C}_i \leftarrow \text{parseResponse}(response)$ 
7:      $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{C}_i$ 
8:   end for
9:   return  $\mathcal{C}$ 
10: end procedure
  
```

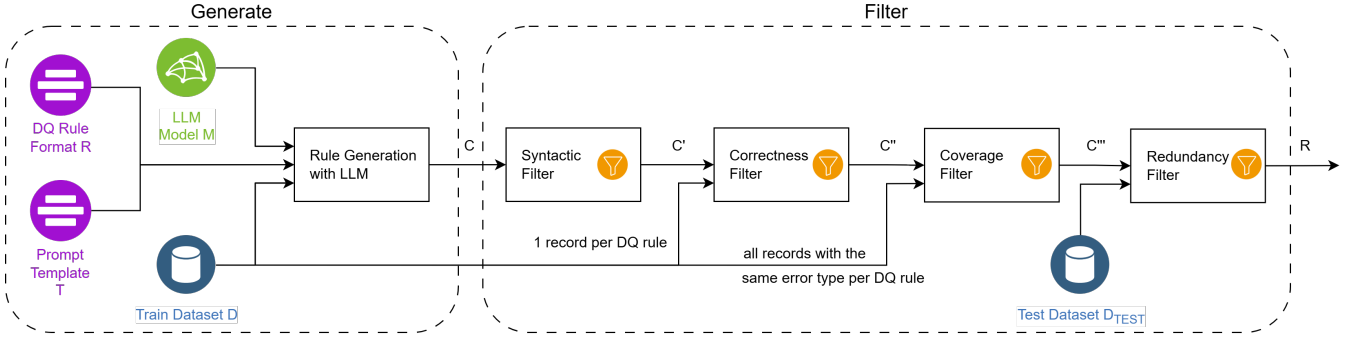


Figure 3: Architecture of LEDQER: an LLM generates candidate rules C , which are progressively refined through four filters $C \supseteq C' \supseteq C'' \supseteq C'''$ until a final rule set R is returned.

Prompt construction. The prompt is a central part of the generation process and consists of the following four sections: the prompt template $\mathcal{P}$ (containing the initial framing and tasks), the input data t_i , the rule syntax $\mathcal{S}$, and the dataset description $Desc$. For each tuple t_i , the function `constructPrompt( $\mathcal{P}, t_i, \mathcal{S}, Desc$ )` instantiates the prompt template into a concrete prompt (cf. Algorithm 1, line 4).

The *initial framing* of the prompt contains overall information such as the LLM’s role (e.g., “You are a GX rule expert.”) and the overall task (e.g., “Generate GX rules to prevent an error in the data sample”).

We support and evaluate three variants of providing the *input data* t_i with increasing difficulty:

- (1) *Dirty & Clean:* The LLM is provided with clean and dirty data. While this scenario is uncommon in practice, it provides the most information to the LLM and serves as an upper bound for our evaluation.
- (2) *Dirty & Type:* In addition to dirty data, the LLM also receives information about the error types present in the columns of a record. This scenario is applicable in case the applied error detection approach returns additional error type information rather than a binary assessment.
- (3) *Just Dirty:* Only dirty data is provided to the LLM. This variant is the most realistic scenario in practice, where no additional data and ground truth are available. Compared to the other input data variants, the LLM needs to detect the error without additional information before generating a rule.

The *rule* section specifies the types of rules the LLM should generate, including information about the DQ tool: (1) A clear description of the application scenario, e.g., focus on generating a single rule set without additional information. (2) Details on the targeted DQ tool, e.g., GX version numbers and rule syntax $\mathcal{S}$ examples. This includes information on generating custom rule functions as well as a list of existing GX rule functions with (e.g., “`ExpectColumnValuesToNotMatchRegex(column: str, regex: str)`”) and without parameters (e.g., “`ExpectColumnValuesToNotMatchRegex`”). Otherwise, there is a high chance to generate incorrect rules.

In the last prompt section, we guide the LLM by defining four consecutive sub-tasks to achieve the overall goal:

- (1) detect the error in the provided data,
- (2) generate rules to detect the identified errors,
- (3) check if the rules are general and fit the given syntax, and
- (4) return the rules in a specific format (in our case JSON).

3.2.2 Filtering pipeline. After generating candidate rules C in the first step of the framework, filters are applied to improve the overall quality of C . In LEDQER, we implement a set of four filters that are sequentially applied to progressively refine the rules and produce a final rule set $C \supseteq C' \supseteq C'' \supseteq C''' \supseteq R$.

Syntactic filter. The syntactic filter verifies whether the generated rules C are executable and match the given DQ rule syntax $\mathcal{S}$. The result is a reduced candidate rule set $C' \subseteq C$ that contains only syntactically correct (i.e., executable) rules. No additional information, such as labeled data, is required for this filter.

Example 3.2. For Example 2.3, this means that the syntactical correctness rate is 0.75 since the fourth rule contains an operation that is not supported in GX (`ColumnNotMatchRegex`) and hence only 3 out of 4 rules are syntactically correct.

Correctness filter. The correctness filter removes rules that fail to filter the tuple from which they were generated. Therefore, we re-apply and verify each generated rule $r \in C'$ on the tuple that has triggered its generation. A rule either recognizes the error (*true positive TP*), fails to recognize the error (*false negative FN*), or mistakenly detects an error in a clean value (*false positive FP*). The filter removes all rules that do not lead to a TP from C' leading to a reduced set of rules $C'' \subseteq C'$.

Example 3.3. Applying this filter to the rules in our Example 2.3 with the data from Table 1 yields an F1 score of 0.5. Specifically, the first rule correctly detects that the PostalCode is wrong (TP). The Street is wrong but not detected by any rule (FN). Rule three incorrectly flags the clean FirstName as wrong, resulting in a false positive (FP).

Coverage filter. To identify rules $r \in C''$ that do not generalize well beyond the original error instance within the observed input tuple t_i , the coverage filter tests each rule against all tuples that share the same error type in the training dataset $\mathcal{D}$. Rules are removed if they achieve a recall above a predefined threshold across these

variants, resulting in $C''' \subseteq C''$. While precision was addressed in correctness filter, the coverage filter focuses exclusively on recall. A high recall indicates strong generalizability across various variants of the same error type. However, a low recall does not necessarily imply that a rule is ineffective, as some errors may require highly specific rules to avoid confusion with correct values. Unfortunately, such rules may exhibit low recall and could be filtered out depending on the chosen threshold. In the current implementation of LEDQER, we do not set a minimum recall threshold to prioritize maximum coverage and avoid the risk of filtering highly specific rules.

Example 3.4. The remaining rule in C'' is applied on tuples 1, 3 and 4 (Table 1). Tuple 2 has a different error and thus will not be used. Tuple 5 shares one error column and error type with tuple 1. However, the LLM did not generate a rule to detect this error successfully in tuple 1, thus the tuple is not selected. As only rule 1 passed the two previous filters, we consider only this rule for the coverage filter. The recall of 0.66 is calculated from

- TP (2): tuple 1 (PostalCode), tuple 3 (PostalCode)
- FP (0):
- FN (1): tuple 4 (PostalCode)

This means the generalizability of the rule is not optimal, as it captures only a subset of error variants. While it successfully detects the dirty tuple 3 (with the PostalCode format "A-<4 digits>"), it fails to identify another dirty tuple (with the PostalCode format "A<4 digits>"). As we do not set a minimum recall threshold, C''' consists of rule 1.

Redundancy filter. The redundancy filter removes duplicated and overlapping rules by using a greedy covering algorithm to reduce the learned rules to a minimal rule set that collectively covers all errors of an error type [12]. We therefore use a separate test dataset $\mathcal{D}_{test}$ to estimate precision and recall. In practice, $\mathcal{D}_{test}$ can be a curated dataset used to validate rule performance. While our evaluation employs large-scale datasets to demonstrate scalability, smaller curated sets are sufficient for practical application. We iteratively select rules with the highest recall among those achieving precision 1, mark their covered tuples, and repeat until all tuples are covered, resulting in the final rule set $\mathcal{R} \subseteq C'''$. Since false positives are costly in practice, precision is the primary filter metric. Unlike the previous filters, the redundancy filter removes overlapping rules in the C''' across an unseen $\mathcal{D}_{test}$. Consequently, its behavior cannot be meaningfully demonstrated using the small-scale Table 1. We therefore defer its illustration to the experimental evaluation in Section 4.5.

4 Experiments

In this section, we experimentally evaluate LEDQER by analyzing each step of the generate-filter framework: the rule generation and the syntactic filter in Section 4.2, the correctness filter in Section 4.3, the coverage filter in Section 4.4, and the redundancy filter in Section 4.5. The experimental setup is described in Section 4.1. As each step targets a different aspect of rule quality, the datasets, evaluation metrics, and number of rules vary by experiment.

4.1 Experimental Setup

All experiments were implemented in Python and are available on GitHub⁷. We ran the experiments on our high-performance computing platform, which features a single AMD EPYC 7643 (48-core, 2.3 GHz) CPU, four NVIDIA A100 GPUs (each with 160GB high-bandwidth memory), and 1TB RAM.

4.1.1 Data quality tool. In our experiments, we create rules for the open source DQ tool Great Expectations(GX), which enables users to create rules ("expectations" in GX terminology) to check whether the validated data contains errors (see Example 2.3 for a syntax example). GX provides multiple predefined rules that can be customized. We have chosen GX in version 1.9.1 (i.e., the latest stable version) for our experiments, since it is freely available, based on Python, has a large and active open-source community, as well as active partnerships with major industry companies such as Snowflake and Databricks [16].

4.1.2 Datasets. We used five datasets for our evaluation, summarized in Table 2. The beers, flights, hospital, and tax datasets are adapted from Mahdavi et al. [25]. The PCI (personal contact information) dataset is synthetically generated from real Austrian addresses [14]. Compared to the original version in [14], we polluted the PCI dataset with additional error types for this study⁸.

Table 2: Dataset characteristics used in our experiments.

Dataset	# Columns	# Dirty Tuples	# Error Types
Beers	10	2,410	1
Flights	7	1,904	3
Hospital	19	407	1
Tax	15	6,000	2
PCI	14	6,000	9

4.1.3 LLMs. To interface with the three selected local LLMs, we utilized the OpenAI Python client⁹. These models were selected based on their rankings in the Chatbot Arena LLM Leaderboard¹⁰ and their compatibility with our computing infrastructure:

- **GLM-4.7-Flash (cyankiwi/GLM-4.7-Flash-AWQ-4bit)¹¹:** a lightweight general-purpose model chosen for its ability to run on weaker hardware, serving as a lightweight baseline.
- **Qwen3-Coder (cpatonn/Qwen3-Coder-30B-A3B-Instruct-AWQ-4bit)¹²:** a code-specialized model that may benefit the generation of syntactically correct rules.
- **Gemma-4 (RedHatAI/gemma-4-31B-it-FP8-Dynamic)¹³:** a high-performance general-purpose model serving as a strong baseline for comparison.

This selection allows us to compare a lightweight model, a code-specialized model, and a high-capacity general-purpose model.

⁷https://github.com/Anna-Christina-Glock/dq_rule_generation

⁸<https://github.com/Anna-Christina-Glock/pci-llm-toolkit/tree/main/data-generator>

⁹<https://github.com/openai/openai-python>

¹⁰<https://lmarena.ai/?leaderboard>

¹¹<https://huggingface.co/cyankiwi/GLM-4.7-Flash-AWQ-4bit>

¹²<https://huggingface.co/cyankiwi/Qwen3-Coder-30B-A3B-Instruct-AWQ-4bit>

¹³<https://huggingface.co/RedHatAI/gemma-4-31B-it-FP8-Dynamic>

Table 3: Syntactic filter: Number of generated rules and the corresponding percentage of executable rules across three LLMs, five $\mathcal{D}$, and three input variants (Dirty & Clean, Dirty & Type, Just Dirty), and two $\mathcal{S}$ variations (with vs. without parameter information).

		GLM-4.7		Gemma-4		Qwen3-Coder	
		no Parameter	Parameter	no Parameter	Parameter	no Parameter	Parameter
Dirty & Clean							
Beers	No. Rules	2,534	2,569	2,984	3,119	3,223	3,137
	% Executable	0.28	0.84	0.99	1.00	0.98	0.94
Flights	No. Rules	2,002	2,748	3,154	902	7,860	7,804
	% Executable	0.45	0.77	0.95	0.93	0.98	0.94
Hospital	No. Rules	468	502	476	484	530	621
	% Executable	0.50	0.80	0.93	0.95	0.93	0.82
PCI	No. Rules	6,539	6,321	6,467	6,835	11,396	11,379
	% Executable	0.37	0.78	0.86	0.93	0.90	0.85
Tax	No. Rules	14,818	15,035	193	2,027	14,251	13,307
	% Executable	0.48	0.85	0.88	0.97	0.94	0.80
Dirty & Type							
Beers	No. Rules	1,977	2,071	2,267	2,398	2,484	2,475
	% Executable	0.32	0.85	1.00	1.00	0.98	0.95
Flights	No. Rules	1,637	1,694	1,965	2,015	2,548	2,836
	% Executable	0.43	0.82	0.99	1.00	0.97	0.90
Hospital	No. Rules	492	543	571	558	481	429
	% Executable	0.43	0.81	0.98	0.99	0.92	0.90
PCI	No. Rules	6,835	7,033	4,076	4,241	16,785	16,691
	% Executable	0.35	0.80	0.91	0.94	0.98	0.91
Tax	No. Rules	4,660	4,858	3,428	1,644	5,626	5,472
	% Executable	0.39	0.81	1.00	1.00	0.99	0.98
Just Dirty							
Beers	No. Rules	2,451	2,466	4,052	4,115	2,686	2,602
	% Executable	0.34	0.76	0.68	0.83	0.89	0.58
Flights	No. Rules	2,503	2,494	4,819	4,892	8,581	8,033
	% Executable	0.46	0.78	0.98	0.99	0.97	0.92
Hospital	No. Rules	746	746	1,489	1,458	1,071	1,048
	% Executable	0.48	0.87	0.98	1.00	0.90	0.84
PCI	No. Rules	9,293	8,887	8,904	9,739	27,745	26,222
	% Executable	0.33	0.78	0.81	0.85	0.99	0.88
Tax	No. Rules	8,535	9,255	8,500	10,059	16,010	16,470
	% Executable	0.44	0.85	0.79	0.93	0.93	0.84

4.2 Evaluation of the Syntactic Filter

Table 3 shows the number of rules generated by the LLMs for each dataset, LLM, input variant, and $\mathcal{S}$ variant (i.e. with/without parameter). Interestingly, the models typically generated more than one rule per tuple. This contradicts our initial expectation of a single rule per tuple, since most tuples contain only one error affecting a single value. This suggests that the LLM generates additional rules to detect other potential errors, which may improve the generalizability of $\mathcal{R}$. However, this also increases the likelihood of overlapping and duplicated rules, further justifying the importance of the filtering pipeline.

To evaluate the syntactic filter, we use the fraction $\frac{|C'|}{|C|}$ of syntactically correct rules as the metric. The findings in Table 3 indicate that the LLMs are reliable in generating executable rules, especially when $\mathcal{S}$ with parameter information is provided. Depending on the model and the dataset, 87.13% to 100% of the rules are executable in that case. This reliability was consistent across different datasets and various model, although the coding-specialized Qwen3-Coder (mean executability = 91%) and the general-purpose Gemma-4 (mean executability = 93%) outperformed the lightweight GLM-4.7-Flash (mean executability = 61%).

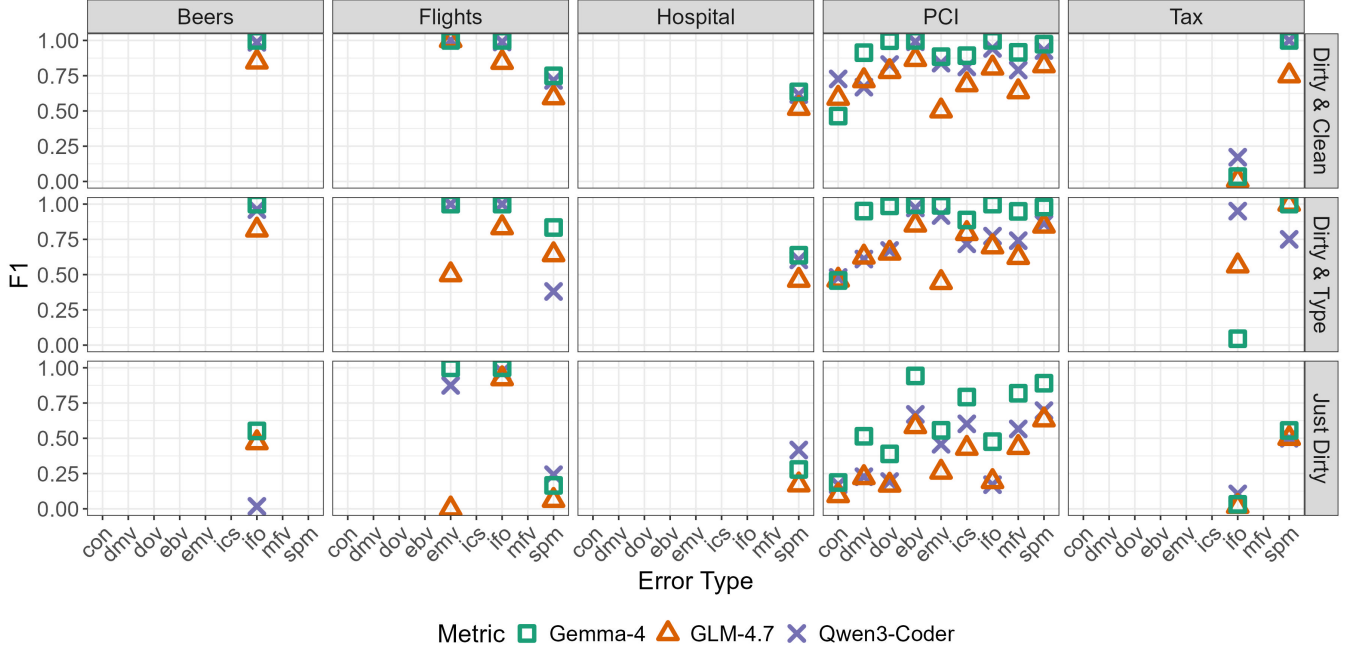


Figure 4: Correctness filter: Aggregated F1-Score per error type across the three LLMs, three input variants, and five datasets.

4.3 Evaluation of the Correctness Filter

The correctness filter is applied to all rules that remain after the syntactic filter. As mentioned in Section 3.2.2, we use F1 to evaluate whether the generated rules correctly identify the error in each tuple.

Figure 4 presents the mean F1 per error type and per types of error information (Just Dirty, Dirty & Type and Dirty & Clean). Because the results of the rules generated with and without parameter information are very similar (max std: 0.087), we will present the mean results for both. Just Dirty performs worst overall, which is expected, as, in addition to creating rules, the dirty values have to be detected. Which is especially difficult cases where business knowledge is needed, for example for the dov or the ifo where the right format might depend of organizational preferences. The results for Dirty & Type and Dirty & Clean demonstrate that it is possible to create successful rules for these error information types as well. The results for Dirty & Type indicate that providing information about the error type and column is enough to increase the overall performance by 0.33 compared to Just Dirty.

Consistent with the observations from the syntactic filter, GLM-4.7-Flash exhibited the lowest performance among the three models. Interestingly, Qwen3-Coder also underperformed relative to Gemma-4. This is likely due to Gemma-4’s better general reasoning capabilities, whereas the specialized coding capabilities that benefited Qwen3-Coder in the syntactic filter were less critical for ensuring the correctness of the rules.

Overall, the rule generation works well across all error types, with a mean F1 of 0.66. Performance varied by error type, with the LLM struggling most with con (F1 = 0.4) and achieving the highest success with ebv (F1 = 0.87). The success with ebv, is likely due to

these errors being easily detectable via regular expressions and only affecting a single value, given how they were generated. In contrast, con involves inconsistencies across multiple columns, necessitating more complex rules.

4.4 Evaluation of the Coverage Filter

Following Section 3.2.2, we use recall as the metric since false positives have been eliminated in previous stages. A high recall indicates good generalizability across different variants of the same error type. To prioritize maximum coverage and avoid the risk of filtering out highly specific rules (as cautioned in Section 3.2.2), no minimum recall threshold was enforced during this stage. Consequently, all candidate rules were retained for the final redundancy filtering process.

Figure 5 shows that regardless of the specific type of error information provided, the rules demonstrate similar generalizability over other instances of the same type, achieving a mean recall of 0.88 (std = 0.16). Contrary to the observations in the syntactic filter and the correctness filter, GLM-4.7-Flash is no longer the lowest-performing model in this stage. The rules generalize similarly across all LLM models (recall std = 0.16), although those generated by Qwen3-Coder often perform the worst of the three. An exception is the con error type, in which Gemma-4 rules exhibit the lowest generalization. The reasons for this may be linked to the semantic complexity of con errors, leading Gemma-4 to produce overly specific rules to avoid specific contradictions.

Even though the generalization works for all error types, performance is notably lower for those with multiple variants, such as dov (recall = 0.58), con (recall = 0.67), and spm (recall = 0.84). For example, for the, column, the LLM created a rule with a regex

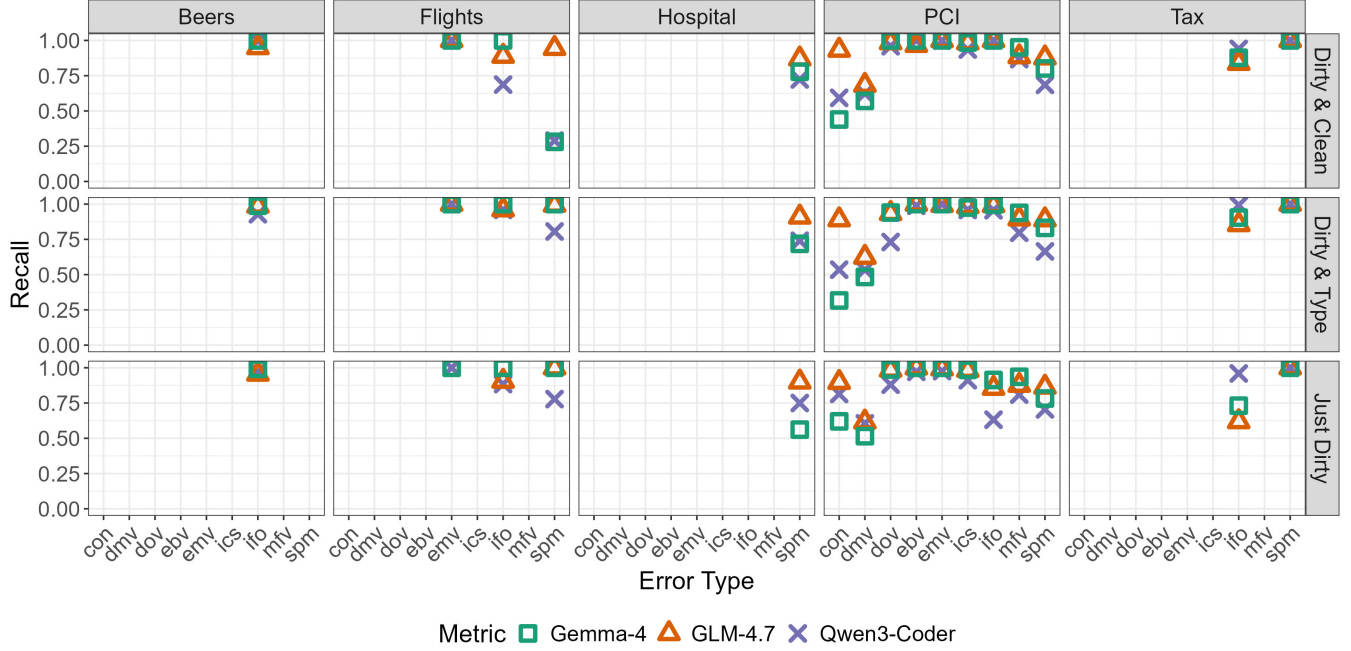


Figure 5: Coverage filter: Aggregated recall per error type across the three LLMs, three input variants, and five datasets.

that correctly detects values starting with a lowercase letter. However, the rule does not generalize to all Austrian city names: cities containing umlauts (e.g., “ö”, “ü”, “ä”), hyphens, or multiple words (e.g., “Hagenberg im Mühlkreis”) are not detected, making the rule overly general but not optimal.

4.5 Evaluation of the Redundancy Filter

Before applying the redundancy filter as described in Section 3.2.2, we evaluate how well each rule in C''' generalizes to a new, unseen dataset $\mathcal{D}_{test}$. For this, recall and precision are computed per rule; see Figure 6 for the results. The figure shows that the rules in C''' generalize well to unseen data, as most of the rules either have a high precision or high recall and a subset exhibits both high precision and high recall. These results are consistent with those reported in the coverage filter, which already demonstrated that the LLM is capable of generating rules that generalize to unseen data but may miss some dirty tuples. Taken together, the individual rule quality suggests that the rule set as a whole should perform well in combination.

We then apply the redundancy filter to C''' and evaluate the resulting final rule set $\mathcal{R}$ against $\mathcal{D}_{test}$. For the beers, flights, hospital, and tax dataset, we aggregate the results over 5 folds, as only one dataset is available. For the PCI we generated a $\mathcal{D}_{test}$ with 1,000,000 tuples 25% of them dirty ones. Figure 7 shows the total number of detected tuples split into TP (dirty tuple) and FP (clean tuple) across different minimum precision thresholds. At a precision of 1 (i.e., no false positives), the rules achieve at least 50% coverage (except for the tax dataset), depending on the error type existing in each dataset. For the tax dataset, a slight increase in false positives (up to 6 FP or 0.007%) results in near-complete error

coverage (99.58%), indicating a favorable trade-off. In contrast, for the flights dataset, relaxing the precision threshold yields comparatively smaller gains of 9% in coverage while introducing a larger number of false positives (+6.7%), suggesting a more conservative threshold selection. This supports our design choice to filter and rank rules by precision, allowing data stewards to adapt the rule set to different error characteristics and tolerance levels. Furthermore, the redundancy filter significantly reduces the number of rules, resulting in the final rule set $\mathcal{R}$ only containing between 2 and 136 rules. Lowering the precision threshold increases coverage at the cost of introducing false positives, exposing a controllable precision–coverage trade-off. This trade-off remains consistent across the different LLMs for the beers, flights, and tax datasets. However, for the hospital and PCI datasets, Gemma-4 exhibits the most favorable trade-off, followed closely by GLM-4.7-Flash and Qwen3-Coder.

4.6 Summary of Results

Our experiments show that an LLM is able to generate syntactically correct rules with executability rates ranging from 87.13% to 100% when provided with both rule names and valid parameters. With purposeful filtering, those rules can be reduced to a small rule set that generalizes well to unseen data.

The results indicate that even providing only the error type and column increases the overall performance (F1 score) by an average of 0.33 compared to providing only dirty values. Generally speaking, the rule generation works well across most error types, yielding a mean F1 score of 0.66, although complex error types such as contradictions showed lower performance (F1 = 0.4). The generalizability of the generated rules is lower for those error types



Figure 6: Precision and recall of each rule measured on an unseen dataset $\mathcal{D}_{test}$ for each of the three LLMs, three data input variants, and five datasets. The results have been ordered by F1 score.

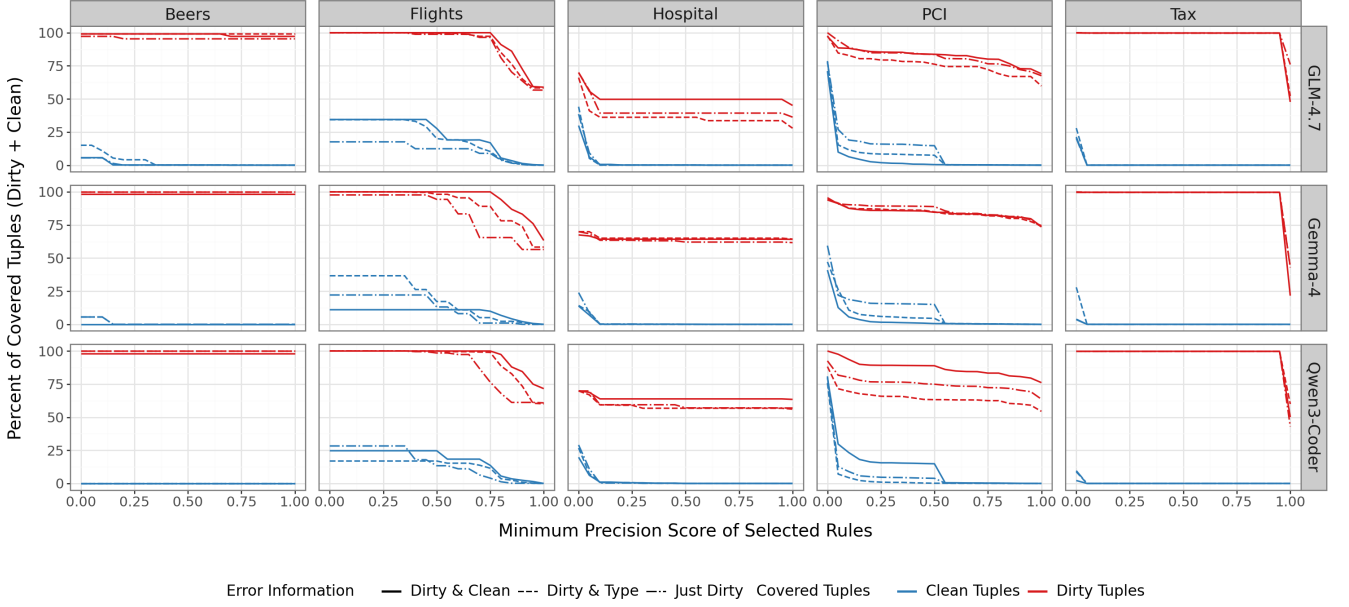


Figure 7: Redundancy filter: Percentage of covered clean and dirty tuples found by the rules in $\mathcal{R}$, calculated for each of the five $\mathcal{D}_{test}$, three input variants and three LLMs with different precision settings for the rules in $\mathcal{C}''$.

with multiple variants, such as *dmv* (recall=0.58) or *con* (recall=0.67), compared to the mean recall of 0.87 across all types.

The evaluation of the redundancy filter demonstrates that at a precision of 1 the reduced rule set achieves at least 50% coverage of the dirty tuples across four of the five unseen datasets. Decreasing this precision threshold can, in some cases, improve coverage significantly. For example, in the tax dataset, introducing only 6

false positives increased the percentage of found dirty values by approximately 70 percentage points to reach near-complete coverage. This highlights a controllable trade-off between precision and recall that a data steward can adjust based on the tolerance for false positives in the respective organization and use case.

5 Related Work

We discuss related work along three lines: rule learning in the machine learning (ML) community, DQ rule discovery in the database community, and LLM-based approaches to DQ rule generation.

Rule learning in the ML community. Learning interpretable error-correcting rules has a long tradition in machine learning in general, and in inductive rule learning in particular. Generalized additive models [17] may be viewed as a family of algorithms that learn multiple layers classifiers where each layer incrementally improves upon the previous ones. In particular boosting-based approaches [11] have a clear focus on learning models that correct classification errors. Patching [22] takes a somewhat different angle in that it investigates the potential of learning interpretable rules that characterize regions of errors in an underlying immutable black-box system. Gao et al. [13] propose a system for learning rules to correct an underlying NLP system. Collectively, these approaches provide important foundations for learning human-understandable rule sets from examples and focus on improving predictive systems by characterizing or correcting model errors. In contrast, LEDQER focuses on generating DQ rules that are executable by a DQ tool to detect errors in structured datasets.

DQ rule discovery in the database community. Within the database community, automatic discovery of DQ rules has been studied primarily in the context of integrity constraints and functional dependencies. Chiang and Miller [5] and Yeh and Puri [44] explored the discovery of DQ rules from data, focusing on conditional functional dependencies and related dependency patterns. These traditional approaches are largely combinatorial, and rely on discovering statistical regularities across large datasets. As a result, they are well suited for structural constraints, but less effective to learn semantic validation rules that depend on contextual information or domain knowledge. To overcome these limitations, recent work explores LLMs for generating higher-level semantic constraints.

LLM-based DQ rule generation. The use of LLMs to generate interpretable rules and features has received increasing attention. Balek et al. [1] propose LLMs for generating tabulated features for natural language documents, and Gottlob [15] propose the use of LLMs for automatic database curation via the Chat2Data framework.

Most closely related to our work are two approaches that also use LLMs to generate DQ rules, which we compare along the lines of our generate-filter formalization in Table 4. Xie et al. [41] follow a requirement-driven approach in which rules are derived from natural-language business requirements and database metadata, tailored to the medical domain of electronic health records. Schneider [38] proposes a template-driven approach where templates specify the tables and operations an LLM should use to generate the rules. Both approaches generate a complete rule set from scratch in a one-time manner, based on what errors the LLM expects rather than on errors observed in the data. In contrast, LEDQER generates rules from observed (i.e., real) errors in the data: for each dirty tuple, LEDQER (1) detects the respective error, (2) suggests a rule to detect this error, and (3) applies the filtering steps outlined in Section 3.2. Rather than generating rules from scratch, LEDQER iteratively refines an existing rule base from a small number of examples, relieving data stewards from manually specifying rules or

Table 4: Comparison of LEDQER with related LLM-based rule generation frameworks. Xie et al. [41] and Schneider [38].

Step	Xie et al.	Schneider	LEDQER
Input	Requirements + Metadata	Template + Metadata	Dirty tuples
Generation	One-time	One-time	Incremental
<i>Filtering</i>			
Syntactic	✓	✓ ^a	✓
Correctness	✗	✓ ^a	✓
Coverage	✗	✗	✓
Redundancy	✓	✓ ^a	✓
Output	SQL rules	Prolog-like rules	GX rules

^a Requires / involves a LLM-as-a-judge.

requirements [38]. Consequently, a direct quantitative comparison with Xie et al. [41] and Schneider [38] is not meaningful, since the three approaches differ in their required input as well as their output (i.e., rule syntax). While Xie et al. [41] require domain-specific business requirements and Schneider [38] require manually specified requirements, LEDQER can be used solely on dirty tuples as input, which reflects a more practical scenario.

6 Conclusion

In this work, we introduced LEDQER, an LLM-based generate-filter framework that generates DQ rules for the enterprise DQ tool GX. Unlike previous works [38, 41], which rely on the manual specification of domain-specific requirements or templates, LEDQER targets a practical setting: a data steward has identified a dirty tuple that passed the validation unnoticed and needs to extend the rule set of GX accordingly. In the *generate* step, an LLM generates candidate rules from the dirty tuple. In the *filter* step, four filters remove low-quality rules. They check the executability and correctness of individual rules, their generalizability to other errors of the same type, and remove redundant rules from the final rule set.

Our evaluation across five datasets demonstrates that LEDQER generates correct and generalizable rules from dirty tuples alone. Providing additional information about the error, such as its type and location, or the clean tuple, further improves the rule quality. Importantly for real-world scenarios, the error type and location alone are sufficient to significantly improve performance. This reduces the need to find clean data for the successful generation of DQ rules. For less complex error types, the dirty tuple alone can still yield sufficient rule sets. Furthermore, LEDQER performs consistently across different LLMs. Interestingly, higher-capacity general-purpose models outperform lightweight or coding-specialized models as error complexity increases. This suggests that general reasoning is more critical for rule correctness than specialized coding proficiency. Finally, we observed an occasional inverse relationship between initial rule correctness and generalizability. This highlights the importance of the generate-filter approach, which filters a wide range of candidate rules.

Future work. Based on our industry partner’s feedback, we plan to systematically evaluate the ability of LLMs to generate natural-language descriptions of the generated rules. Such descriptions also allow non-technical users to understand and use these rules and automate documentation processes. We consider this direction promising since our manual reviews revealed that the LLM occasionally produced useful rule descriptions without explicit prompting. Furthermore, we aim to explore methods for automatically detecting the error type of a given dirty tuple. Our results already demonstrate its potential: providing the error type significantly improves the rule generation performance.

Acknowledgments

The research was supported by the Austrian ministries BMIMI, BMWET and the State of Upper Austria in the frame of the SCCH COMET competence center INTEGRATE (FFG 892418) and by the “ICT of the Future” project QuanTD (no. 898626), as well as by the Austrian Science Fund FWF (10.55776/COE12). RedHatAI/gemma-4-31B-it-FP8-Dynamic and Claude Pro were used to generate code and code documentation for the experimental evaluation, and to revise and proofread parts of the paper. All ideas and the scientific content of this paper originate from the authors. Where text was AI-assisted, it was rewritten and verified by the authors, who take full responsibility for the content of this work.

Artifacts

The code implementing LEDQER is available in a public GitHub repository¹⁴. The code to generate the PCI dataset is also available in a public GitHub repository¹⁵, and the other four (polluted and labeled) datasets are also publicly available¹⁶. The LLMs used during the experiments are available via HuggingFace¹⁷.

References

- [1] Vojtěch Balek, Lukáš Sýkora, Vilem Sklenák, and Tomáš Kliegr. 2025. LLM-based feature generation from text for interpretable machine learning. *Machine Learning* 114, 11 (2025), 241. doi:10.1007/S10994-025-06867-1
- [2] Divya Bhadauria, Hazar Harmouch, Felix Naumann, Divesh Srivastava, and Lisa Ehrlinger. 2026. A Catalog of Data Errors. arXiv:2604.09277 [cs.DB] <https://arxiv.org/abs/2604.09277>
- [3] Jan-Micha Bodensohn, Ulf Brackmann, Liane Vogel, Anupam Sanghi, and Carsten Binnig. 2025. Unveiling Challenges for LLMs in Enterprise Data Engineering. *Proc. VLDB Endow.* 19, 2 (2025), 196–209.
- [4] Roshan Chandru, Provia Kadusabe, Asifa Mehmood Qureshi, Shubham Sharma, and Abhishek Kaushik. 2025. Large Language Models (LLMs) in Medical Error Detection and Correction: A Comprehensive Review. *Next-Gen Healthcare: AI-Powered Medical Innovations* (2025), 149–173.
- [5] Fei Chiang and René J Miller. 2008. Discovering data quality rules. *Proceedings of the VLDB Endowment* 1, 1 (2008), 1166–1177.
- [6] Melody Chien, Divya Radhakrishnan, and Sue Waite. 2025. *Magic Quadrant for Augmented Data Quality Solutions*. Technical Report. Gartner. <https://www.gartner.com/en/documents/6246519>
- [7] João Brainer Clares de Andrade, Marconny Alexandre Oliveira de Medeiros Cavalcante, Thiago Luis Marques Lopes, João Marcos Secundino Treigher, Mateus Dutra Balsells, Júlia Lima Vasconcelos, Lis Cavalcante Monteiro, and Déborah Danna da Silveira Mota. 2026. Discovery of data quality issues in electronic health records: profound consequences for critical care medicine applications – a systematized review. *Critical Care* 30, 1 (2026). doi:10.1186/s13054-025-05677-0
- [8] Lisa Ehrlinger and Wolfram Wöß. 2022. A Survey of Data Quality Measurement and Monitoring Tools. *Frontiers Big Data* 5 (2022), 850611.
- [9] European Parliament and Council of the European Union. 2016. General Data Protection Regulation (GDPR). <https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:02016R0679-20160504>
- [10] European Parliament and Council of the European Union. 2024. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L series. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689>
- [11] Yoav Freund and Robert E. Schapire. 1997. A decision-theoretic generalization of on-line learning and an application to boosting. *J. Comput. System Sci.* 55, 1 (1997), 119–139.
- [12] Johannes Fürnkranz. 1999. Separate-and-Conquer Rule Learning. *Artificial Intelligence Review* 13, 1 (1999), 3–54.
- [13] Tong Gao, Shivang Singh, and Raymond J. Mooney. 2022. Towards Automated Error Analysis: Learning to Characterize Errors. In *Proceedings of the 35th International Florida Artificial Intelligence Research Society Conference (FLAIRS)*. Florida Online Journals, Hutchinson Island, Jensen Beach, Florida. doi:10.32473/FLAIRS.V35I.130632
- [14] Anna-Christina Glock, Christine Dominka-Kiss, Philipp Korom, and Lisa Ehrlinger. 2025. Detecting and Cleaning Errors in Personal Contact Information with Large Language Models. In *Proceedings of the 2nd International Workshop on Data Acquisition and Analytics (DATA@VLDB)*. VLDB Endowment. https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/DATA/DATAI25_12.pdf
- [15] Georg Gottlob. 2024. Enhancing Data Precision with Large Language Models: Analyzing Failures and Innovating Database Curation. In *Proceedings of the 32nd Symposium of Advanced Database Systems (CEUR Workshop Proceedings, Vol. 3741)*. CEUR-WS.org, Villasilimus, Italy, 1–2.
- [16] Great Expectations. 2026. Great Expectations’ Official Partners. <https://greatexpectations.io/partners/>
- [17] Trevor Hastie and Robert Tibshirani. 1986. Generalized Additive Models. *Statist. Sci.* 1, 3 (1986), 297–310. doi:10.1214/ss/1177013604
- [18] Anders Haug, Frederik Zachariassen, and Dennis van Liempd. 2011. The costs of poor data quality. *J. Ind. Eng. Manag.* 4, 2 (2011). doi:10.3926/jiem.2011.v4n2.p168-193
- [19] Ihab F. Ilyas and Felix Naumann. 2022. Data Errors: Symptoms, Causes and Origins. *IEEE Data Eng. Bull.* 45, 1 (2022), 4–9.
- [20] João Marcelo Borovina Josko, Marcio Katsumi Oikawa, and João Eduardo Ferreira. 2016. A Formal Taxonomy to Improve Data Defect Description. In *DASFAA (Lecture Notes in Computer Science, Vol. 9645)*. Springer, 307–320. doi:10.1007/978-3-319-32055-7_25
- [21] Philipp Jung, Sebastian Jäger, Nicholas Chandler, and Felix Biessmann. 2025. Towards Realistic Error Models for Tabular Data. *ACM J. Data Inf. Qual.* 17, 4 (2025), 28:1–28:27.
- [22] Sebastian Kauschke and Johannes Fürnkranz. 2018. Batchwise Patching of Classifiers. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI-18)*. 3374–3381. <https://www.aaai.org/ocs/index.php/AAAI/AAAI18/paper/view/16099>
- [23] Won Kim, Byoung-Ju Choi, Eui-Kyeong Hong, Soo-Kyung Kim, and Doheon Lee. 2003. A Taxonomy of Dirty Data. *Data Mining and Knowledge Discovery* 7, 1 (2003), 81–99. doi:10.1023/A:1021564703268
- [24] David Loshin. 2002. Rule-based data quality. In *Proceedings of the Eleventh International Conference on Information and Knowledge Management (McLean, Virginia, USA) (CIKM ’02)*. Association for Computing Machinery, New York, NY, USA, 614–616. doi:10.1145/584792.584894
- [25] Mohammad Mahdavi, Ziawash Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data (Amsterdam, Netherlands) (SIGMOD ’19)*. Association for Computing Machinery, New York, NY, USA, 865–882. doi:10.1145/3299869.3324956
- [26] Sedit Mohammed, Lukas Budach, Moritz Feuerpfeil, Nina Ihde, Andrea Nathansen, Nele Noack, Hendrik Patzlaff, Felix Naumann, and Hazar Harmouch. 2025. The effects of data quality on machine learning performance on tabular data. *Information Systems* 132 (2025), 102549.
- [27] Lapo Mori, Bryan Richardson, Tamim Saleh, Richard Sellschop, and Ian Wells. 2023. Clearing data-quality roadblocks: Unlocking AI in manufacturing. <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/clearing-data-quality-roadblocks-unlocking-ai-in-manufacturing/>. [Accessed 28-01-2026].
- [28] H. Müller and J.C. Freytag. 2005. *Problems, Methods, and Challenges in Comprehensive Data Cleansing*. Humboldt-Univ. zu Berlin.
- [29] Avanika Narayan, Ines Chami, Laurel J. Orr, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *Proc. VLDB Endow.* 16, 4 (2022), 738–746.
- [30] Paulo Oliveira, Fátima Rodrigues, and Pedro Rangel Henriques. 2005. A Formal Definition of Data Quality Problems. In *Proceedings of the 2005 International Conference on Information Quality (MIT ICIQ Conference)*. MIT.
- [31] R. Pearson. 2006. The problem of disguised missing data. In *SIGKDD Explorations Newsletter*, Vol. 8. ACM, 83–92.

¹⁴https://github.com/Anna-Christina-Glock/dq_rule_generation

¹⁵<https://github.com/Anna-Christina-Glock/pci-llm-toolkit/tree/main/data-generator>

¹⁶https://github.com/Anna-Christina-Glock/dq_rule_generation

¹⁷<https://huggingface.co/models>

- [32] Leo L. Pipino, Yang W. Lee, and Richard Y. Wang. 2002. Data Quality Assessment. *Commun. ACM* 45, 4 (2002), 211–218. doi:10.1145/505248.506010
- [33] Raghav R, Adarsh Prakash Vemali, Darpan Aswal, Rahul Ramesh, and Ayush Bhupal. 2025. ScottyPoseidon at SemEval-2025 Task 8: LLM-Driven Code Generation for Zero-Shot Question Answering on Tabular Data. In *Proceedings of the 19th International Workshop on Semantic Evaluation (SemEval-2025)*. Association for Computational Linguistics, Vienna, Austria, 2197–2204. <https://aclanthology.org/2025.semeval-1.285/>
- [34] Erhard Rahm and Hong Hai Do. 2000. Data Cleaning: Problems and Current Approaches. *IEEE Data Eng. Bull.* 23, 4 (2000), 3–13.
- [35] Thomas C. Redman. 1998. The Impact of Poor Data Quality on the Typical Enterprise. *Commun. ACM* 41, 2 (1998), 79–82. doi:10.1145/269012.269034
- [36] Tobias Rehberger, Thomas Hütter, Lisa Ehrlinger, and Wolfram Wöß. 2026. Evaluating Data Quality Tools: Measurement Capabilities and LLM Integration. *arXiv preprint arXiv:2604.09163* (2026). <https://arxiv.org/abs/2604.09163>
- [37] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proceedings of the VLDB Endowment* 10, 11 (2017), 1190–1201. doi:10.14778/3137628.3137631
- [38] Alexander Schneider. 2025. *Template-guided rule generation and evaluation for data quality using Large Language Models*. Master's thesis. TU Wien.
- [39] Ikbal Taleb, Mohamed Adel Serhani, Chafik Bouhaddioui, and Rachida Dssouli. 2021. Big data quality framework: a holistic approach to continuous quality management. *J. Big Data* 8, 1 (2021), 76. doi:10.1186/S40537-021-00468-0
- [40] Richard Y. Wang and Diane M. Strong. 1996. Beyond Accuracy: What Data Quality Means to Data Consumers. *Journal of Management Information Systems* 12, 4 (1996), 5–33. doi:10.1080/07421222.1996.11518099
- [41] Shuyang Xie, Hailing Cai, Yaoqin Sun, and Xudong Lv. 2025. LLM-DQR: Large language model-based automated generation of data quality rules for electronic health records. *J. Biomed. Inform.* 172, 104951 (2025), 104951.
- [42] Devansh Yadav and Shouvick Mondal. 2025. Evaluating pre-trained Large Language Models on zero shot prompts for parallelization of source code. *J. Syst. Softw.* 230, 112543 (2025), 112543. doi:10.1016/j.jss.2025.112543
- [43] Qinchen Yang, Zhiqing Hong, Dongjiang Cao, Haotian Wang, Zejun Xie, Tian He, Yunhuai Liu, Yu Yang, and Desheng Zhang. 2025. AddrLLM: Address Rewriting via Large Language Model on Nationwide Logistics Data. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1* (Toronto ON, Canada) (KDD '25). ACM, New York, NY, USA, 2756–2767.
- [44] Peter Z. Yeh and Colin A. Puri. 2010. An Efficient and Robust Approach for Discovering Data Quality Rules. In *22nd IEEE International Conference on Tools with Artificial Intelligence, ICTAL*. IEEE Computer Society, 248–255. doi:10.1109/ICTAL.2010.43